

## Bridge Day 16 STUDENT WORKBOOK

Indexed Mutation and Independent Exam 2 Rehearsal

**Learning sequence:** Predict - Trace - Explain - Test - Interpret

Monday, July 27, 2026 • 50 points • longitudinal template 07242026\_v1

|             |  |            |  |                |  |
|-------------|--|------------|--|----------------|--|
| <b>Name</b> |  | <b>UIN</b> |  | <b>Section</b> |  |
|-------------|--|------------|--|----------------|--|

### Daily target and independence boundary

Design, mutate, summarize, and test a complete list-processing program from requirements alone.

|                         |                                                                                            |
|-------------------------|--------------------------------------------------------------------------------------------|
| <b>Allowed tools</b>    | All Exam 2 tools: loops, conditions, list reads, append, len, and indexed assignment       |
| <b>Support supplied</b> | Only the requirements, constraints, and test obligations are supplied.                     |
| <b>Student decides</b>  | The entire algorithm, variable inventory, loop form, mutation order, summaries, and tests. |
| <b>Scaffold level</b>   | Independent construction; no planning questions and no algorithm steps.                    |

### Evidence and scoring

| <b>Evidence</b>            | <b>Points</b> | <b>Secure work shows</b>                                                    |
|----------------------------|---------------|-----------------------------------------------------------------------------|
| Integrated trace           | 8             | intermediate state, condition results, exact output, and meaning            |
| Diagnosis and repair       | 6             | actual, intended, cause, repair, and a discriminating test                  |
| Complete program and tests | 36            | coherent executable logic, required output, assumptions, and defended tests |

**Task 1. Integrated trace - 8 points**

---

Produce one complete mutation ledger and exact final output.

```
values = [-2, 7, 14, 3]
changed = 0

for index in range(len(values)):
    original = values[index]
    if original < 0:
        values[index] = 0
    elif original % 2 == 1:
        values[index] = original * 2
    else:
        continue
    changed = changed + 1

print(values, changed)
```

**Your task**

1. For every index, record original value, selected branch, final value, list state, and changed; then give exact output.

**Task 2. Diagnose and repair - 6 points**

---

The intent is to replace every negative value, including the last item, and count each replacement.

```
values = [4, -2, 8, -5]
changed = 0

for index in range(len(values) - 1):
    if values[index] < 0:
        values[index] = 0
        changed = changed + 1

print(values, changed)
```

**Your task**

1. Give actual and intended output, identify the boundary defect, repair it, and name a test that would fail if the last index were skipped again.

---

**Task 3. Construct a complete program - 36 points**

---

**Program specification**

- Use temperatures = [18, -4, 31, 22, 50].
- Mutate every value below 0 to 0 and every value above 40 to 40.
- Count how many positions change.
- Using the final mutated values, compute total, average, and the count from 20 through 40 inclusive.
- Print the final list, changed count, total, average, and in\_range count in that order on separate lines.
- Do not use sum, min, max, comprehensions, slicing, or helper functions. Provide two tests with expected results, including a no-change case and a first/last-change case.

**Program workspace**

**Task 3 continued. Test and defend - included in 36 points**

---

Continue code if needed. Required tests, expected results, and brief defense

## Bridge Day 16 STUDENT WORKBOOK

### VIP Evidence Handoff

**Learning sequence:** Predict - Trace - Explain - Test - Interpret

Contract 07a04826c975 • longitudinal template 07242026\_v1

### Why engineers use this page

Rehearse indexed traversal, filtering, compound Boolean evidence, and the Day 6 Independent Program Studio inside the announced loop/list Exam 2 boundary; do not introduce function structure today.

**Decision boundary:** Code is useful only when its stored state, visible result, assumptions, units, limits, and tests support a defensible engineering interpretation.

### Construct readiness before independent work

**New authoring tools:** string traversal, list traversal, list index read, list index write, string index read, slice expression, membership test, parallel list indexing, compound filter

**Carried authoring tools:** assignment, print call, arithmetic expression, input call, int conversion, float conversion, f string, comparison expression, boolean operator, if elif else, for loop, range call, append call, len call

**Supplied-code exposure only:** none

An exposure-only construct may be traced in complete supplied code, but the independent task will not require students to invent it before its authoring day.

### Notebook cells and task constructs

| Activity | Work                      | Stable cells                             | Required authoring constructs                                                                               |
|----------|---------------------------|------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| 18.25    | Reverse Values            | 18-25-work / 18-25-transfer / 18-25-cold | append call, for loop, range call                                                                           |
| 18.26    | Values At Or Above Cutoff | 18-26-work / 18-26-transfer / 18-26-cold | append call, comparison expression, for loop                                                                |
| 18.27    | Feasible Workstations     | 18-27-work / 18-27-transfer / 18-27-cold | append call, boolean operator, f string, float conversion, for loop, input call, int conversion, range call |

### Readiness evidence

- For Activity 18.25, write the exact index sequence used to visit every value in reverse order without changing the original list.
- For Activities 18.26 and 18.27, state the keep condition and name one boundary case that must be tested before Exam 2.

### Timing and help trigger

Record a first prediction before running. If you still lack an executable plan after about 8 focused minutes, use the linked ThinkCSPy refresher or the supplied model. If two attempts fail for the same named requirement, write the exact mismatch and ask a peer mentor or instructor a targeted question. Timing is diagnostic evidence, not a speed grade. **Start or elapsed minutes:** \_\_\_\_\_

**My help trigger:**

## Engineering Evidence Record

Complete one record from a model, transfer, or Independent Program Studio build. Generic statements are not sufficient; use actual values, a real revision, and one concrete next test.

| Evidence field          | Record |
|-------------------------|--------|
| Prediction              |        |
| Observed Result         |        |
| Revision Reason         |        |
| Engineering Meaning     |        |
| Units Or Not Applicable |        |
| Assumption              |        |
| Model Limit             |        |
| Next Test               |        |
| Help Trigger            |        |
| Minutes Used            |        |

### Notebook and submission procedure

1. Attempt, predict, run, inspect, and revise before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those visibly labeled Studio cells are scored for independent mastery.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.